

DATA STRUCTURE AND ALGORITHM

Project Proposal

Title: Maze Explorer

Submitted by:

Name: Wasif Ali Reg No: 245187

Name: Jawad Mustafa Reg No: 245195

Name: Abrar Ahmed Reg No: 245193

Submitted to:

SIR M. KAMRAN

Department of Computer Science (BS-CS)

Air University Aerospace and Aviation Campus Kamra

1. Project Title

Maze Explorer, A DSA-Based Text Maze Game in C++

2. Project Description

Maze Explorer is a text-based adventure game built in C++ where the player navigates through a maze to reach the exit while avoiding obstacles and collecting items. The maze is generated and solved using core Data Structures and Algorithms covered in the course.

The project demonstrates practical applications of graphs, BFS, DFS, stacks, queues, and hashing by implementing features such as random maze generation, shortest-path auto-solving, move undoing, and inventory management.

This project simulates a real-world scenario of pathfinding, similar to robot navigation, GPS shortest-path systems, and AI decision-making.

The objective is to create an interactive system that helps understand how DSAs work behind real applications. It provides educational value by visually showing how algorithms like BFS/DFS operate.

3. Functional Requirements

A. Core Features

1. **Maze Representation**
    - Maze stored in a 2D grid and internally converted to a graph.
    - Walls, paths, player, exit, treasures represented using characters.
2. **Player Movement**
    - Player moves using W/A/S/D.
    - Movement validated using graph neighbor checks.
3. **Random Maze Generation**
    - Uses Depth-First Search (DFS) algorithm.
    - Ensures a solvable maze with proper paths.
4. **Shortest Path Finder (Auto-Solve)**
    - Uses Breadth-First Search (BFS) to calculate shortest path from player to exit.
    - Highlights the computed path in the maze.

5. **Undo Movement**
    - Previous positions stored in a Stack.
    - Player can undo moves using a single key.
6. **Inventory Management**
    - Treasures stored using Hash Table (unordered_map).
    - Fast lookup and counting of collected items.
7. **File Handling**
    - Save and load maze from text files.
    - Optional save of scores or player progress.

B. Integration of Data Structures & Algorithms

Data Structure / Algorithm	How It's Used
Graph (Adjacency List/Matrix)	Represents maze connectivity
DFS	Maze generation
BFS	Shortest path solver
Stack	Undo player moves
Queue	BFS implementation
Hash Table	Inventory management
Array / Vector	Storing maze map
File Handling	Saving / loading maps
AVL / BST	Leaderboard high scores